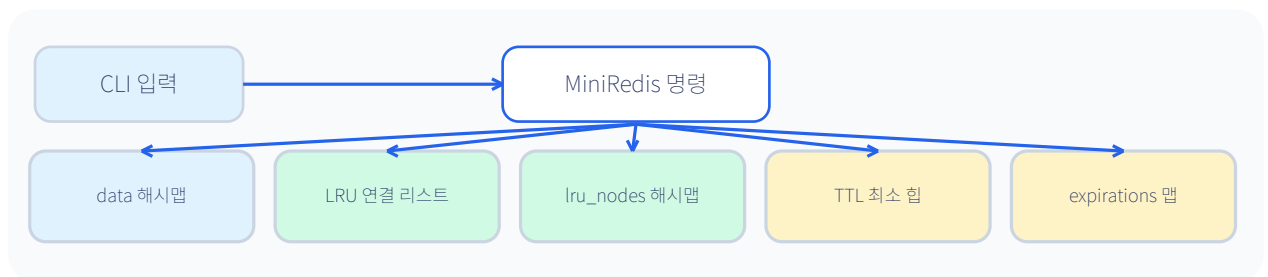


Mini Redis 개념서

해시맵 · 이중 연결 리스트 · 최소 힙으로 이해하는 LRU와 TTL



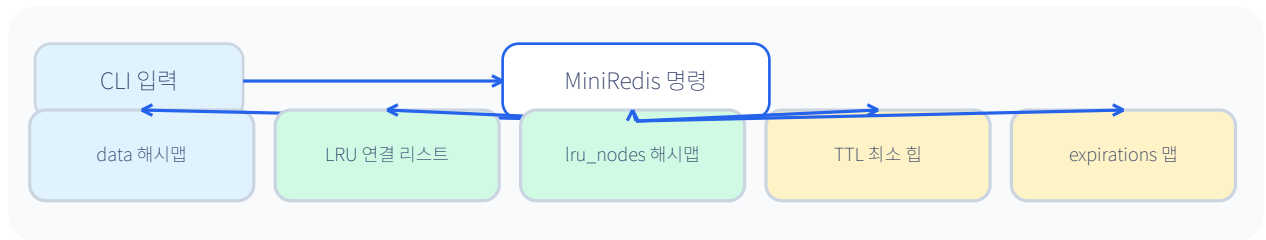
학습 목표	코드를 읽고 평가 문항 17개에 자신의 말로 답하기
구현 범위	필수 명령 10개 + CLI, 보너스 과제 제외
읽는 순서	전체 그림 → 자료구조 → 명령 흐름 → 평가 Q&A

기준 문서: 미션 - AI 도구 학습.pdf / B5-2_평가.pdf

Python 3.8+ · CLI only · dict/set/collections 미사용

1. 전체 구조와 핵심 용어

Mini Redis는 키로 값을 찾는 작은 메모리 저장소다. 빠른 조회, 사용 순서, 만료 순서는 서로 다른 문제이므로 자료구조도 역할을 나눠 가진다.



구조	쉬운 비유	이 코드의 역할	복잡도
해시맵	이름으로 사물함 번호 찾기	키→값·노드·만료 시각 조회	평균 $O(1)$
이중 연결 리스트	앞뒤 사람을 모두 아는 줄	최근 사용 순서와 tail 제거	$O(1)$
최소 힙	가장 이른 알람이 맨 위	가장 빠른 만료 찾기	peek $O(1)$
LRU	오래 안 쓴 것부터 버리기	메모리 제한 초과 시 eviction	평균 $O(1)/키$

$O(1)$ 은 데이터가 늘어도 한 번의 작업량이 거의 일정하다는 뜻이다. 해시맵은 평균 $O(1)$ 이며, 연결 리스트는 대상 노드를 이미 알고 있을 때 $O(1)$ 이다. 힙 삽입·삭제는 $O(\log n)$ 이다.

구현 파일

- doubly_linked_list.py: Node와 6개 필수 리스트 메서드
- hash_map.py: 직접 만든 해시, 체이닝, 로드 팩터 기반 resize
- min_heap.py: push/pop/peek/size와 두 heapify
- mini_redis.py: 명령, 메모리, LRU, TTL을 결합

2. 자료구조를 코드로 이해하기

2.1 이중 연결 리스트

Node는 prev, next, data를 가진다. 리스트는 head와 tail도 기억한다. 삭제할 노드의 양옆 링크만 바꾸므로 전체를 순회하지 않는다.

```
def remove_node(self, node):
    if node.prev is None:
        self.head = node.next
    else:
        node.prev.next = node.next

    if node.next is None:
        self.tail = node.prev
    else:
        node.next.prev = node.prev
```

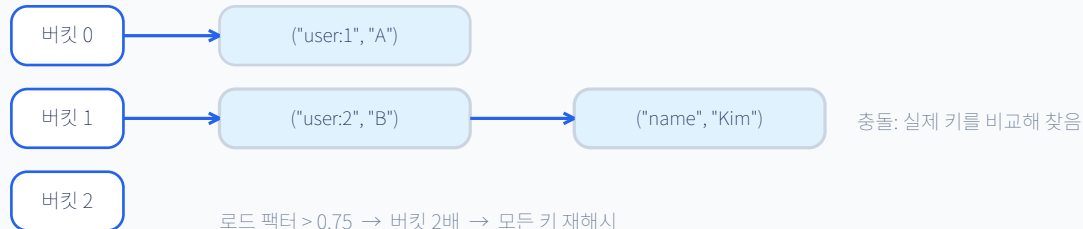
중요: 노드를 찾는 작업까지 리스트로 하면 $O(n)$ 이다. LRU에서는 lru_nodes 해시맵이 키로 노드를 바로 알려준다.

2.2 체이닝 해시맵

해시는 문자열 키를 숫자 인덱스로 바꾸는 규칙이다. 이 구현은 UTF-8 바이트를 차례로 섞고 버킷 수로 나눈 나머지를 사용한다.

```
value = 0
for byte in key.encode("utf-8"):
    value = (value * 31 + byte) & 0xFFFFFFFF
return value % self.capacity
```

해시 인덱스가 같으면 한 버킷의 연결 리스트에 이어 붙인다

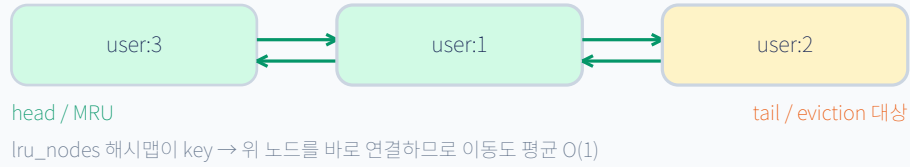


충돌은 서로 다른 키가 같은 인덱스를 얻는 현상이다. 버킷마다 이중 연결 리스트를 두고 실제 키가 같은지 비교한다. $\text{count} / \text{capacity}$ 가 0.75를 넘으면 버킷을 2배로 만들고 모든 키를 다시 해시한다.

3. LRU와 TTL

3.1 해시맵 + 이중 연결 리스트로 만드는 LRU

LRU: 최근 사용은 앞(head), 가장 오래된 사용은 뒤(tail)

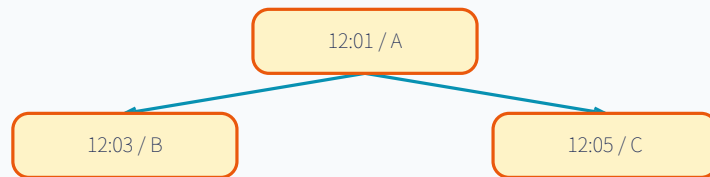


해시맵은 키의 노드를 평균 O(1)에 찾고, 연결 리스트는 그 노드를 O(1)에 앞으로 옮긴다. 둘 중 하나만 있으면 순서 또는 빠른 노드 조회를 잃는다.

```
def _touch(self, key):
    node = self.lru_nodes.get(key)
    if node is None:
        node = self.lru.insert_front(key)
        self.lru_nodes.put(key, node)
    else:
        self.lru.move_to_front(node)
```

3.2 최소 힙으로 관리하는 TTL

TTL 최소 힙: 가장 이른 expire_at이 루트에 온다



최신 expirations 값과 다르면 오래된 힙 항목으로 보고 무시한다 (lazy deletion)

TTL(Time To Live)은 키가 살아 있을 남은 시간이다. 최소 힙의 루트에는 가장 작은 expire_at이 온다. EXPIRE를 다시 설정해 생긴 과거 힙 항목은 expirations 해시맵의 최신 값과 비교해 무시한다. 이것이 lazy deletion(지연 삭제)이다.

DEL과 만료 삭제는 공통 _delete_key를 호출한다. 데이터, used_memory, LRU 노드, 최신 TTL을 한 번에 정리해 구조 사이의 불일치를 막는다.

4. 명령 전체 흐름



4.1 SET과 eviction

- 만료된 키를 먼저 정리하고 새 키+값의 UTF-8 바이트 크기를 계산한다.
- 한 항목만으로 maxmemory를 넘으면 저장하지 않고 OOM을 반환한다.
- 덮어쓰기면 이전 크기를 빼고 새 크기를 더하며, 기존 TTL은 지운다.
- SET한 키를 LRU 앞으로 옮기고, 제한을 넘는 동안 tail부터 제거한다.

```
while self.maxmemory > 0 and self.used_memory > self.maxmemory:
    oldest_key = self.lru.tail.data
    self._delete_key(oldest_key)
    self.evicted_keys += 1
```

4.2 GET

```
if self._delete_if_expired(key) or not self.data.contains(key):
    return "(nil)"
value = self.data.get(key)
self._touch(key)
return self._quote(value)
```

만료 또는 없음이면 즉시 끝하므로 LRU가 갱신되지 않는다. 값 반환에 성공한 경우에만 `_touch`가 실행된다.

4.3 메모리 공식

$used_memory = \sum (len(utf8(key)) + len(utf8(value)))$. 노드 · 포인터 · 버킷 오버헤드는 과제 공식에 따라 제외한다.
maxmemory가 0이면 무제한이다.

5. 출력 규칙과 평가 Q&A

상황	출력
SET 성공	OK
GET 없음 또는 만료	(nil)
DEL / EXISTS / EXPIRE	(integer) 1 또는 (integer) 0
TTL: 키 없음 / 만료 없음 / 만료 있음	-2 / -1 / 남은 초
한 항목이 maxmemory보다 큼	OOM, 저장하지 않음
잘못된 명령 / 인자 / 정수	(error) ...

질문 문구는 B5-2_평가.pdf 항목 1-4를 그대로 옮겼다. 항목 5는 보너스 문제이므로 지시대로 제외했다.

항목 1 · 기능 동작

Q1

String 타입 기본 동작: SET, GET, DEL, EXISTS, DBSIZE, KEYS 명령어가 모두 정상 동작하는가?

네. MiniRedis가 여섯 메서드를 제공하고 execute가 명령 이름과 인자 수를 검사해 연결한다. SET/GET은 LRU도 갱신하고, DBSIZE/KEYS는 먼저 만료 키를 정리한다.

Q2

LRU 자동 제거: maxmemory 설정 후 제한 초과 시 가장 오래된 키가 자동 제거되는가?

네. 최근 사용 키는 리스트의 head, 가장 오래된 키는 tail에 있다. SET 뒤 제한을 넘으면 tail부터 _delete_key로 지우며 제한 이하가 될 때까지 반복한다.

Q3

메모리 정보 확인: INFO memory에서 used_memory/maxmemory/evicted_keys가 규칙에 맞게 출력되는가?

네. used_memory는 모든 키와 값의 UTF-8 바이트 길이 합이고 구조 오버헤드는 제외한다. maxmemory는 설정값, evicted_keys는 LRU로 제거한 누적 개수다.

항목 1 · 기능 동작 (계속)

Q4

TTL 관리: EXPIRE/TTL 규칙이 동작하며 만료된 키가 적절히 제거되는가?

네. EXPIRE는 최신 만료 시각을 해시맵과 최소 힙에 저장한다. TTL은 없음 -2, 만료 설정 없음 -1, 설정 있음 남은 초를 반환한다. 접근하거나 전체 정리를 할 때 지난 키를 삭제한다.

Q5

에러 처리: 잘못된 명령/인자/정수 오류/OOM이 표준 형식으로 출력되는가?

네. execute가 알 수 없는 명령과 인자 수 오류를 구분한다. 정수 변환 실패·음수 maxmemory는 정수 오류를, 한 항목이 제한보다 크면 OOM을 반환한다.

항목 2 · 기본 자료구조

Q6

이중 연결 리스트의 노드 구조(prev, next, data)와 핵심 메서드들이 O(1)로 동작하도록 어떻게 구성했는지 설명할 수 있는가?

Node가 앞·뒤 노드와 데이터를 기억하고 리스트가 head, tail을 기억한다. 삽입·삭제·이동은 대상 노드와 이웃의 링크 몇 개만 바꾼다. 대상 노드를 이미 아는 조건에서 O(1)이다.

Q7

해시맵에서 직접 설계한 해시 함수가 어떤 입력을 받아 어떤 과정을 거쳐 인덱스를 만드는지 설명할 수 있는가?

문자열 키를 UTF-8 바이트열로 바꾼다. 각 바이트마다 $value = value * 31 + byte$ 를 계산하고 32비트로 제한한다. 마지막 값을 버킷 수로 나눈 나머지가 인덱스다.

Q8

충돌 해결을 체이닝 방식으로 어떻게 구현했는지(버킷 내부 구조 선택 포함) 설명할 수 있는가?

버킷 배열의 각 칸에 DoublyLinkedList를 둔다. 같은 인덱스를 받은 (key, value)들을 그 리스트에 연결한다. 조회는 해당 버킷만 순회하며 실제 키가 같은 노드를 찾는다.

Q9

로드 팩터 0.75 초과 시 “버킷 2배 확장”을 어떤 절차로 수행하는지 설명할 수 있는가?

새 키를 넣은 뒤 $count / capacity$ 가 0.75보다 큰지 본다. 크면 capacity를 2배로 만들고 빈 버킷 배열을 만든다. 기존 모든 노드를 돌며 새 capacity로 인덱스를 다시 계산해 삽입한다.

항목 3 · LRU, TTL, 명령 흐름

Q10

LRU 구현에서 “해시맵 + 이중 연결 리스트”가 각각 어떤 역할을 하고, 왜 둘 다 필요한지 설명할 수 있는가?

해시맵 `lru_nodes`는 키로 리스트 노드를 평균 $O(1)$ 에 찾는다. 리스트는 최근 사용 순서를 유지하며 앞 이동과 뒤 제거를 $O(1)$ 에 한다. 해시맵만으로는 순서를 모르고, 리스트만으로는 노드를 찾는 데 $O(n)$ 이 걸리므로 둘 다 필요하다.

Q11

$O(1)$ LRU 달성 원리를 “조희(해시) + 갱신(리스트 이동)” 관점에서 설명할 수 있는가?

`_touch`가 해시맵에서 키의 노드를 평균 $O(1)$ 에 조희하고 `move_to_front`가 링크만 바꿔 $O(1)$ 에 앞으로 옮긴다. 가장 오래된 키도 `tail`로 즉시 찾는다.

Q12

TTL 관리에 힙을 사용한 이유(가장 빠른 만료를 빠르게 찾는 성질)를 설명할 수 있는가?

최소 힙 루트에는 가장 작은 `expire_at`, 즉 가장 먼저 만료될 항목이 있다. 다음 만료 확인은 $O(1)$, 새 만료 추가와 루트 제거는 $O(\log n)$ 이므로 모든 키를 매번 $O(n)$ 으로 훑지 않아도 된다.

Q13

메모리 초과 시 eviction 흐름을 단계별로 설명할 수 있는가? (`used_memory` 산정/갱신, LRU 제거, `evicted_keys` 증가 포함)

새 키+값의 UTF-8 바이트 크기를 계산한다. 기존 키면 이전 크기를 빼고 새 크기를 더한다. SET한 키를 LRU 앞에 둔 뒤 초과하면 `tail` 키를 모든 구조에서 지우고 그 크기를 `used_memory`에서 빼며 `evicted_keys`를 증가시킨다. 제한 이하가 될 때까지 반복한다.

Q14

GET 명령어 전체 흐름을 순서대로 설명할 수 있는가? (TTL 확인→삭제 여부→값 반환→LRU 갱신 조건)

키의 만료 시각을 먼저 확인한다. 지났다면 모든 관련 구조에서 삭제하고 `(nil)`을 반환한다. 만료되지 않았지만 키가 없어도 `(nil)`이다. 존재할 때만 값을 읽고 LRU 맨 앞으로 옮긴 다음 따옴표로 감싼 값을 반환한다.

항목 4 · 확장 사고

Q15

“만약 LRU 대신 LFU정책을 구현한다면 자료구조를 어떻게 변경해야 하는가?”에 합리적으로 답변할 수 있는가?

LFU는 가장 오래 안 쓴 키가 아니라 사용 횟수가 가장 적은 키를 지운다. 각 키의 빈도를 GET/SET 때 증가시켜야 한다. 효율적으로는 키→(값, 빈도, 노드) 해시맵, 빈도→같은 빈도의 키 목록 해시맵, 현재 최소 빈도 값이 필요하다. 같은 빈도끼리는 연결 리스트로 최근 사용 순서를 정한다.

Q16

“데이터가 10만 건으로 늘어나면 현재 구조에서 병목이 될 수 있는 부분은 어디이며, 어떻게 개선할 수 있는가?”에 답변할 수 있는가?

해시 분산이 나쁘면 한 버킷 리스트가 길어져 조회가 $O(n)$ 에 가까워질 수 있다. 더 좋은 해시와 적절한 초기 capacity가 필요하다. resize의 전체 재해시 비용은 점진적 rehash로 나눌 수 있다. EXPIRE를 반복하면 lazy deletion 항목이 힙에 쌓이므로 필요할 때 힙을 재구성할 수 있다. KEYS는 모든 키를 출력하므로 본질적으로 $O(n)$ 이다.

Q17

“used_memory에 자료구조 오버헤드까지 포함하는 모델로 바꾸면 무엇이 달라지고, 공정한 비교/채점을 위해 어떤 보정이 필요할지” 설명할 수 있는가?

노드 객체, 포인터, 버킷 배열, 힙 항목까지 세면 같은 키·값이어도 구현과 Python 버전에 따라 값이 달라지고 더 빨리 eviction된다. 공정한 채점을 위해 실행 환경과 측정법, 포함 항목, 중복 참조 처리 규칙을 고정해야 한다. 또는 언어별 고정 오버헤드 상수와 허용 오차를 정할 수 있다.

답변 공식

어떤 구조인가 → 왜 필요한가 → 어느 코드가 담당하는가 → 시간 복잡도는 무엇인가 순서로 설명한다. GET과 SET을 손으로 따라가며 used_memory, LRU의 head/tail, TTL 값이 어떻게 바뀌는지 확인하면 전체 문항이 연결된다.

완료 체크

필수 요구사항만 구현 · 보너스 제외 · 평가 질문 17개 답변 · 자동 테스트 9개 통과